Universidad INVENIO

Licenciatura en Ingeniería en Tecnologías de Información
y Comunicación Empresarial

KNOWVAULT

Enterprise Knowledge Management System

with Artificial Intelligence

Project Documentation

Team Members: Adrián José Caldera Alfaro
Kevin Mauricio García Gutiérrez
Sebastián González Tabares

Course: Programación II

Professor: Jaffette Solano Arias

Date: [Date]

Table of Contents

1. Introduction

1.1 Project Overview

KnowVault is an enterprise-grade knowledge management system designed to enable organizations to store, organize, and query internal documentation through an intelligent web-based platform. The system targets small and medium-sized enterprises that handle large volumes of documentation distributed across multiple locations, including policies, manuals, reports, contracts, and technical guides.

The platform combines traditional document management capabilities with artificial intelligence, allowing users to retrieve information from stored documents using natural language queries. Rather than manually searching through files, users interact with a conversational interface powered by a Retrieval-Augmented Generation (RAG) architecture, which ensures that responses are grounded exclusively in the organization's own knowledge base.

1.2 Problem Statement

Organizations of all sizes face a persistent challenge in managing institutional knowledge. As companies grow, the volume of internal documentation increases substantially, often resulting in fragmented repositories where critical information becomes difficult to locate. Employees spend significant amounts of time searching for specific policies, procedures, or technical details, which reduces overall productivity and creates inconsistencies in decision-making.

Traditional document management systems offer storage and basic search functionality; however, they lack the ability to interpret user intent or provide contextual answers. Full-text search engines can locate documents containing specific keywords, but they cannot synthesize information from multiple sources or respond to complex questions that require understanding of context.

KnowVault addresses this problem by integrating an AI-driven query system that processes natural language questions, retrieves the most relevant document fragments through semantic search, and generates comprehensive answers that reference the original source materials.

1.3 General Objective

The general objective of this project is to develop a responsive web application that centralizes enterprise documentation and provides intelligent query capabilities through the integration of artificial intelligence, specifically using a Retrieval-Augmented Generation architecture and the Anthropic Claude API.

1.4 Specific Objectives

- Design and implement a normalized relational database schema capable of supporting document storage, categorization, text fragmentation, and query history tracking.

- Develop a backend application using Java and Spring Boot that follows the Model-View-Controller architectural pattern and provides RESTful API endpoints for document management, user authentication, and AI-powered queries.
- Build a responsive frontend interface using HTML5, CSS3, and JavaScript that provides an intuitive user experience across desktop, tablet, and mobile devices.
- Implement a Retrieval-Augmented Generation pipeline that processes uploaded documents into searchable chunks, generates vector embeddings for semantic search, and uses the AI model to produce context-aware responses.
- Ensure system security through encrypted password storage, token-based authentication, prepared SQL statements, and server-side file validation.

1.5 Project Scope

The scope of KnowVault encompasses a fully functional web application with the following capabilities: user authentication and role-based authorization, document upload with support for PDF, DOC, DOCX, and TXT formats, automatic document categorization, a complete RAG pipeline for AI-assisted queries, query history with source tracking, and a statistics dashboard. The application is designed to be responsive across desktop, tablet, and mobile viewports.

1.6 Development Team

Team Member	Role	Responsibilities
Sebastián González Tabares	Project Lead / Backend Developer	Team coordination, user management backend module, MVC implementation, data validation, error handling
Adrián José Caldera Alfaro	Backend Developer	AI module architecture, RAG system design, document processing workflow, API integration
Kevin Mauricio García Gutiérrez	Database Engineer	Database design and normalization, SQL scripts, frontend templates and CSS design system, project documentation

2. System Requirements

2.1 Functional Requirements

RF-001: User Authentication

Priority: High

The system shall allow users to register and log in using email and password credentials. The system shall validate the email format during registration, store passwords using bcrypt encryption, and generate a JSON Web Token (JWT) upon successful authentication. Tokens shall expire after 24 hours.

RF-002: Document Management

Priority: High

The system shall allow users to upload, view, search, and delete documents. Supported file formats include PDF, DOC, DOCX, and TXT, with a maximum file size of 10 MB per upload. Users shall be able to assign a title, description, and one or more categories to each document. The document listing view shall support both grid and list layouts, with filtering by category and sorting by date.

RF-003: AI-Powered Queries

Priority: High

The system shall enable users to submit questions in natural language and receive answers generated exclusively from the content of stored documents. Each response shall include references to the source documents and specific text fragments used. The system shall record the response time for each query.

RF-004: Query History

Priority: Medium

The system shall maintain a complete history of all queries submitted by each user. Each history entry shall include the date, question, response, and source references. Users shall be able to search through their history, delete individual entries, or clear the entire history.

RF-005: Document Categorization

Priority: Medium

The system shall support predefined categories (Technical, Business, Legal, Financial, Human Resources, and Other) and allow a many-to-many relationship between documents and categories. Users shall be able to filter documents by category, and administrators shall be able to create new categories.

RF-006: Dashboard

Priority: Medium

The system shall display a dashboard summarizing user activity, including total documents uploaded, total queries performed, number of categories in use, the five most recent documents, and the ten most recent activities.

RF-007: RAG Processing Pipeline

Priority: High

Upon document upload, the system shall automatically extract text content, divide it into chunks of approximately 300 to 500 words, and generate vector embeddings for each chunk. The semantic search mechanism shall identify the most relevant chunks for each user query, and the system shall track which chunks were used in generating each response.

RF-008: Responsive Interface

Priority: High

The application shall provide a responsive layout with defined breakpoints at 480px (mobile), 768px (tablet), and 1024px (desktop). Mobile devices shall display a hamburger menu for navigation, horizontal scrolling for tables, and touch-optimized controls.

2.2 Non-Functional Requirements

RNF-001: Security

All passwords shall be encrypted using bcrypt with a cost factor of 10 or higher. JWT tokens shall be signed with a secret key. All database queries shall use prepared statements to prevent SQL injection. Uploaded files shall be validated by MIME type, not solely by file extension. All API endpoints shall require valid authentication.

RNF-002: Performance

AI query responses shall be delivered within five seconds. The dashboard shall load within two seconds. Document search operations shall complete within one second. The system shall support a minimum of 50 concurrent users.

RNF-003: Scalability

The database shall support a minimum of 10,000 documents. The chunking system shall handle documents of up to 100 pages. The storage architecture shall be designed to allow future migration to cloud storage services such as Amazon S3.

RNF-004: Usability

The user interface shall follow modern design principles inspired by platforms such as Notion and iCloud. The primary typeface shall be Inter (Google Fonts), the primary brand color shall be #0066FF, and all interactive elements shall include smooth animations with durations between 150 and 250 milliseconds.

RNF-005: Maintainability

The Java codebase shall follow Spring Boot conventions. CSS shall use variables for colors and spacing. Complex code sections shall include inline comments. Version control commits shall be descriptive, and documentation shall be updated with every major change.

RNF-006: Compatibility

The application shall be compatible with Chrome 90+, Safari 14+, Firefox 88+, and Edge 90+. The interface shall render correctly on screen resolutions ranging from 320px (iPhone SE) to 3840px (4K displays).

2.3 Technical Constraints

- The backend must be developed in Java 17 using the Spring Boot framework.
- The database engine must be MySQL 8.0, managed through XAMPP during development.
- Database access must use JDBC with Spring JDBC (JdbcTemplate) for direct SQL execution.
- The frontend must be rendered server-side using Thymeleaf templates with HTML5, CSS3, and vanilla JavaScript.
- AI integration must be performed through the Anthropic Claude API for response generation and the OpenAI API for embedding generation.

3. System Architecture

3.1 Architectural Overview

KnowVault follows a three-tier architecture comprising a presentation layer (frontend), a business logic layer (backend), and a data persistence layer (database). The system is built using the Model-View-Controller (MVC) design pattern provided by Spring Boot, where Models represent the data entities and business objects, Views are rendered through Thymeleaf HTML templates, and Controllers manage HTTP request routing and response orchestration.

The general data flow proceeds as follows: the user interacts with the frontend interface in a web browser, which communicates with the Spring Boot backend via HTTP requests. The backend processes these requests through its controller, service, and repository layers, interacting with the MySQL database through JDBC. For AI-powered queries, the backend additionally communicates with external AI APIs to generate embeddings and produce natural language responses.

3.2 Technology Stack

3.2.1 Backend Technologies

Technology	Version	Purpose
Java	17+	Primary programming language
Spring Boot	3.x	Web application framework (MVC, dependency injection, auto-configuration)
Spring MVC	3.x	HTTP request handling and routing
Spring JDBC	3.x	Database access layer via JdbcTemplate
Maven	3.8+	Build automation and dependency management
MySQL Connector/J	8.0.33	JDBC driver for MySQL database connectivity
Thymeleaf	3.x	Server-side HTML template engine
Jakarta Bean Validation	-	Input validation annotations (@NotBlank, @Email, @Size)

3.2.2 Frontend Technologies

Technology	Version	Purpose
HTML5	-	Page structure and semantic markup
CSS3	-	Styling, layout, responsive design, and animations
JavaScript (ES6+)	-	Client-side interactivity and DOM manipulation
Google Fonts (Inter)	-	Primary typeface for the user interface
SVG	-	Scalable vector icons throughout the interface

3.2.3 External Services

Service	Purpose
Anthropic Claude API	Natural language response generation from document context
OpenAI Embeddings API	Vector embedding generation for semantic search

3.2.4 Development Tools

Tool	Purpose
Git / GitHub	Version control and remote repository hosting
XAMPP	Local MySQL server and Apache for development
phpMyAdmin	Visual database administration
Draw.io	Technical diagram creation (ER, flowcharts)
VS Code / IntelliJ IDEA	Integrated development environments

3.3 Technology Justification

Spring Boot was selected as the backend framework due to its comprehensive ecosystem for building Java web applications, including built-in support for dependency injection, auto-configuration, embedded servers, and seamless integration with JDBC and Thymeleaf. Its MVC pattern enforces a clear separation of concerns that aligns with the project's architectural requirements.

MySQL 8.0 was chosen as the relational database engine for its reliability, widespread adoption, and strong support for complex queries, indexing, and foreign key constraints. The use of JDBC via Spring's JdbcTemplate provides direct control over SQL execution while maintaining compatibility with the Spring ecosystem.

Thymeleaf was adopted as the server-side template engine because it integrates natively with Spring Boot, supports natural HTML templating, and eliminates the need for a separate frontend framework, simplifying deployment and reducing complexity.

The RAG architecture was selected because it ensures that AI-generated responses are grounded in actual organizational documents, preventing hallucinations and maintaining factual accuracy. The combination of embedding-based semantic search with a large language model provides a powerful yet controlled approach to knowledge retrieval.

3.4 Layer Description

3.4.1 Presentation Layer (Frontend)

The frontend consists of five primary Thymeleaf-based HTML templates: Dashboard, Upload Document, Documents List, Query History, and Ask AI. A complete CSS design system of

approximately 2,500 lines provides consistent styling across all pages, including a glassmorphism-inspired dark aesthetic. The interface is fully responsive, with a collapsible sidebar navigation that transforms into a hamburger menu on mobile devices.

3.4.2 Business Logic Layer (Backend)

The backend is organized into distinct layers following Spring Boot conventions. The Controller layer handles HTTP routing through both view controllers (for page rendering) and REST controllers (for API endpoints). The Service layer encapsulates business logic, acting as an intermediary between controllers and data access. The Repository layer manages database interactions through JDBC, implementing data access objects for each entity. The AI integration layer manages communication with external APIs for embedding generation and response synthesis.

3.4.3 Data Persistence Layer (Database)

The persistence layer is built on MySQL 8.0 with a schema consisting of seven normalized tables. The database supports the full lifecycle of document management, from user authentication through document storage, categorization, text chunking, and query tracking. All tables use the InnoDB storage engine with UTF-8 (utf8mb4) character encoding, and referential integrity is enforced through foreign key constraints with appropriate cascade rules.

4. System Design

4.1 Entity-Relationship Diagram

The database was designed following Third Normal Form (3NF) principles to minimize redundancy and ensure data integrity. The schema consists of seven tables with clearly defined relationships:

- USERS (1) → (N) DOCUMENTS: A single user may upload multiple documents.
- USERS (1) → (N) QUERY_HISTORY: A single user may submit multiple queries.
- DOCUMENTS (N) ↔ (M) CATEGORIES: Documents and categories share a many-to-many relationship, resolved through the DOCUMENT_CATEGORIES junction table.
- DOCUMENTS (1) → (N) DOCUMENT_CHUNKS: Each document is split into multiple text fragments for the RAG pipeline.
- QUERY_HISTORY (1) → (N) QUERY_SOURCES: Each query response may reference multiple document sources.
- QUERY_SOURCES (N) → (1) DOCUMENTS and DOCUMENT_CHUNKS: Each source entry references a specific document and optionally a specific chunk.

The complete Entity-Relationship diagram is maintained as a Draw.io file (entity-relationship.drawio) in the project repository under the database/diagrams/ directory.

4.2 Database Schema

The following subsections describe each table in the database schema, including field definitions, data types, constraints, and indexing strategies.

4.2.1 Users Table

The users table stores account information for all system users, including administrators and regular users.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique user identifier
email	VARCHAR(255)	NO	UNI	-	Unique email address
password_hash	VARCHAR(255)	NO	-	-	Bcrypt-encrypted password
full_name	VARCHAR(255)	YES	-	NULL	Full name

role	ENUM('admin','user')	YES	-	'user'	System role
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Registration date
updated_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Last modification

4.2.2 Categories Table

The categories table stores the predefined document classification labels used for organizing uploaded files.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique category ID
name	VARCHAR(100)	NO	UNI	-	Category name
description	TEXT	YES	-	NULL	Category description
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Creation date

4.2.3 Documents Table

The documents table stores metadata for all uploaded files, including ownership, file system references, and access timestamps.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique document ID
user_id	INT	NO	FK	-	Uploading user reference
title	VARCHAR(255)	NO	-	-	Document title
description	TEXT	YES	-	NULL	Optional description
file_name	VARCHAR(255)	NO	-	-	Original filename
file_path	VARCHAR(500)	NO	-	-	Server storage path
file_size	BIGINT	NO	-	-	File size in bytes
mime_type	VARCHAR(100)	NO	-	-	MIME type

upload_date	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Upload date
last_accessed	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Last access timestamp

4.2.4 Document Categories Table (Junction)

The document_categories table resolves the many-to-many relationship between documents and categories through a composite primary key.

Field	Type	Null	Key	Default	Description
document_id	INT	NO	PRI, FK	-	Document reference
category_id	INT	NO	PRI, FK	-	Category reference

4.2.5 Document Chunks Table

The document_chunks table is central to the RAG pipeline. It stores text fragments extracted from documents along with their corresponding embedding vectors for semantic search.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique chunk ID
document_id	INT	NO	FK	-	Parent document
chunk_text	TEXT	NO	-	-	Extracted text fragment
chunk_index	INT	NO	-	-	Position in document
embedding	JSON	YES	-	NULL	Vector for semantic search
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Processing date

4.2.6 Query History Table

The query_history table records all AI interactions, preserving both the user's question and the system's response along with performance metrics.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique query ID
user_id	INT	NO	FK	-	Querying user

query_text	TEXT	NO	-	-	Original question
response_text	TEXT	NO	-	-	AI-generated response
response_time_ms	INT	YES	-	NULL	Response time (ms)
created_at	TIMESTAMP	YES	-	CURRENT_TIMESTAMP	Query timestamp

4.2.7 Query Sources Table

The query_sources table provides full traceability of AI responses by recording which documents and specific text chunks contributed to each answer, along with their relevance scores.

Field	Type	Null	Key	Default	Description
id	INT	NO	PRI	AUTO_INCREMENT	Unique source ID
query_id	INT	NO	FK	-	Parent query reference
document_id	INT	NO	FK	-	Source document
chunk_id	INT	YES	FK	NULL	Specific chunk used
relevance_score	DECIMAL(3,2)	YES	-	NULL	Relevance (0.00–1.00)

4.3 Use Case Description

The system supports the following primary use cases, organized by actor:

Regular User: Log in and log out of the system; upload documents with metadata and category assignment; view, search, filter, and delete personal documents; submit natural language queries to the AI system; view AI-generated responses with source references; browse and manage personal query history; view dashboard statistics.

Administrator: All regular user capabilities; create and manage document categories; manage user accounts.

4.4 Navigation Flow

The application follows a linear navigation structure accessible through a persistent sidebar menu. Upon successful authentication at the Login page, the user is redirected to the Dashboard. From the sidebar, the user may navigate to any of the five main views: Dashboard, Upload Document, My Documents, Ask AI, and Query History. On mobile devices, the sidebar collapses into a hamburger menu that slides in from the left edge of the screen.

5. Implementation

5.1 Module Overview

The implementation of KnowVault was divided among the three team members according to their assigned roles. The following subsections describe the modules developed by each contributor.

5.2 Database Module (Kevin García)

The database module encompasses the complete data persistence layer of the system. The design process began with requirements analysis to identify the entities, attributes, and relationships necessary to support all system functionalities, from user management to AI-powered query tracking.

5.2.1 Schema Design

The database schema was designed following Third Normal Form (3NF) to eliminate redundancy and ensure referential integrity. Seven tables were created using the InnoDB storage engine with utf8mb4 character encoding to support international characters. The schema includes strategic indexing on frequently queried columns such as email, user_id, upload_date, and created_at to optimize query performance.

Two SQL scripts were produced: create_database.sql for the complete schema definition including tables, constraints, and indexes; and seed_data.sql for initial test data comprising sample users, categories, documents, chunks, and query records.

5.2.2 Database Connection

A dedicated DatabaseConnection class was implemented to manage JDBC connectivity to the MySQL instance. This utility class follows the singleton pattern, maintaining a single persistent connection throughout the application lifecycle. Connection parameters are configured through the Spring Boot application.properties file, specifying the JDBC URL, credentials, and driver class.

5.2.3 Technical Diagrams

Four technical diagrams were created using Draw.io to document the system's data flows and structure: an Entity-Relationship diagram depicting all seven tables and their relationships; an Authentication Flow diagram illustrating the login process from credential submission through JWT generation; a Document Upload Flow diagram covering file validation, storage, metadata recording, and background chunk processing; and a RAG Query Flow diagram detailing the semantic search and AI response generation pipeline.

5.3 Frontend Module (Kevin García)

The frontend module comprises five complete Thymeleaf-based HTML templates and a comprehensive CSS design system. All templates were developed with a consistent design

language inspired by modern cloud platforms, featuring a glassmorphism-influenced dark aesthetic with semi-transparent elements, subtle blur effects, and smooth animations.

5.3.1 Templates Developed

- **dashboard.html:** Main landing page displaying statistical cards (total documents, total queries, categories count), a recent documents section, and an activity timeline.
- **upload-document.html:** Document upload interface featuring a drag-and-drop zone, file preview panel, and a metadata form with fields for title, category selection, and description.
- **documents-list.html:** Document management view with toggle between grid and list layouts, category filtering, sort ordering, and per-document actions (view, download, delete).
- **ask-ai.html:** Conversational AI interface with a chat-style message area, text input with send button, a source documents panel, and suggested question cards.
- **history.html:** Query history view presenting past interactions in a timeline format, with search, date filtering, and individual or bulk deletion capabilities.

5.3.2 CSS Design System

The CSS design system spans approximately 2,500 lines and provides a complete set of design tokens, component styles, and responsive rules. The system employs CSS custom properties for colors, spacing, and typography, ensuring consistency and facilitating future theming. The primary color palette uses #0066FF as the brand color, with supporting shades for success (#10B981), warning (#F59E0B), and error (#EF4444) states. Responsive breakpoints are defined at 480px, 768px, and 1024px, with layout adjustments at each threshold including sidebar collapse, grid reconfiguration, and touch-optimized control sizing.

5.4 User Management Backend Module (Sebastián González)

The user management module implements the backend infrastructure for creating, viewing, and deleting user accounts within the system. This module was developed following a layered architecture that separates concerns across model, repository, service, and controller layers.

5.4.1 Model Layer

The User entity class encapsulates user data attributes including id, name, email, and password. Input validation is enforced through Jakarta Bean Validation annotations: @NotBlank ensures that required fields are not empty, @Email verifies proper email formatting, and @Size enforces minimum character length requirements on specific fields.

5.4.2 Repository Layer

The data access layer consists of a UserRepository interface and its JdbcUserRepository implementation. The repository uses Spring JDBC's JdbcTemplate to execute parameterized SQL queries for CRUD operations. This approach provides direct control over SQL statements while leveraging Spring's connection management and exception translation.

5.4.3 Service Layer

The service layer contains the business logic that mediates between the controllers and the repository. It encapsulates validation rules, data transformation logic, and transaction boundaries, ensuring that the controller layer remains focused on HTTP request handling.

5.4.4 Controller Layer

The controllers handle HTTP requests routed to /users, /users/new, and user deletion endpoints. Each controller communicates with the service layer to process requests and returns the corresponding Thymeleaf views populated with dynamic data.

5.4.5 Error Handling

A global exception handler was implemented using Spring's @ControllerAdvice annotation. This component intercepts uncaught exceptions across all controllers and redirects users to a custom error page, preventing the display of default server error pages and improving the overall user experience.

5.5 AI Module (Adrián Caldera)

The AI module is responsible for the entire Retrieval-Augmented Generation pipeline, from document ingestion through response generation. The architecture ensures that all AI-generated answers are grounded exclusively in the organization's stored documents.

5.5.1 Document Processing Workflow

When a document is uploaded to the system, the AI module initiates a multi-step processing pipeline. First, the text content is extracted from the uploaded file using appropriate parsing libraries. The extracted text is then divided into smaller segments called chunks, each containing approximately 400 to 500 characters. These chunks are stored in the document_chunks table along with their positional index and page number reference. Finally, each chunk is converted into a numerical vector representation (embedding) using the AI embedding API, and the resulting vector is stored in the embedding_vector column for subsequent semantic search operations.

5.5.2 Question Processing Workflow

When a user submits a question through the Ask AI interface, the system executes the following pipeline: the question text is first converted into an embedding vector using the same embedding model. The system then performs a semantic similarity search by comparing the question embedding against all stored chunk embeddings, selecting the most relevant fragments based on cosine similarity scores. The selected chunks are assembled into a context block, which is sent alongside the user's question to the AI language model via API. The model is explicitly instructed to answer only using the provided context; if the information is not found in the documents, the system returns a predefined fallback message.

5.5.3 Response Storage

Once the AI generates a response, the system persists the complete interaction in the query_history table, recording the original question, the generated response, and the response time in milliseconds. The documents and specific chunks that contributed to the response are recorded in the query_sources table with their associated relevance scores, providing full traceability for every AI-generated answer.

5.5.4 Safety and Reliability

The AI module incorporates several safety measures. The model is configured to use only the retrieved document context when generating responses, preventing the generation of information not present in the knowledge base. The system includes protection against prompt injection attempts that could override system instructions. When the requested information cannot be found in the stored documents, the system returns a standardized fallback message rather than fabricating an answer.

6. Conclusions

6.1 Achievements

The KnowVault project achieved the successful development of a comprehensive enterprise knowledge management platform that integrates document storage, categorization, and AI-powered query capabilities. The following milestones were completed:

- A fully normalized database schema consisting of seven tables was designed and implemented, supporting the complete data lifecycle from user registration through document processing, AI querying, and response tracking.
- A layered backend architecture was established using Spring Boot with clearly separated controller, service, and repository layers, including a functional user management module with CRUD operations, input validation, and global error handling.
- Five professional frontend templates were developed with a cohesive design system, delivering a responsive user interface that adapts across desktop, tablet, and mobile viewports.
- The RAG architecture was designed and documented, including the complete document processing pipeline (text extraction, chunking, embedding generation) and the query processing pipeline (question embedding, semantic search, context assembly, and AI response generation).
- Comprehensive project documentation was produced, including four technical diagrams, a complete data dictionary, SQL scripts, and this project report.

6.2 Challenges Encountered

- **Frontend-backend integration:** Coordinating the development of frontend templates with backend endpoints required careful communication among team members. The decision to develop the frontend first provided a clear visual target for backend implementation but required iterative adjustments as API contracts were finalized.
- **Responsive design complexity:** Adapting the interface to function consistently across a wide range of screen sizes required extensive CSS work and testing on multiple devices. A mobile-first development approach would have simplified this process.
- **Database design iterations:** The initial database schema underwent several revisions to accommodate the requirements of the RAG system, particularly the document_chunks and query_sources tables, which were not part of the original design and required careful integration with existing relationships.
- **Version control discipline:** Maintaining backup copies and using descriptive Git commits proved essential for recovering from breaking changes, reinforcing the importance of consistent version control practices in collaborative development.

6.3 Lessons Learned

- Thorough upfront database design significantly reduces the need for structural changes during development and facilitates parallel work across the team.
- A frontend-first development approach provides a clear visual reference that improves team communication and accelerates backend development.
- Continuous documentation during development is more effective and accurate than retrospective documentation at the end of the project.
- Layered architecture with clear separation of concerns enables each team member to work independently on their assigned modules while maintaining system coherence.

6.4 Future Work

The following enhancements are planned for subsequent development phases:

Short-term (two weeks): Completion of the full REST API, initial integration with the Claude API for basic AI query functionality, and end-to-end integration testing between frontend and backend.

Medium-term (one month): Full RAG pipeline implementation with embedding generation, deployment to a test server, database performance optimization through additional indexing, and UX refinements based on user feedback.

Long-term (three months): Advanced role-based permissions, administrative dashboard, support for additional file formats (XLS, PPT), integration with cloud storage services (Google Drive, Dropbox), real-time notifications, dark mode theme, and multi-language support.

7. References

The following sources were consulted during the design, development, and documentation of the KnowVault project:

Anthropic. (2026). Claude API Documentation. https://docs.anthropic.com

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

MySQL. (2026). MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

OpenAI. (2026). Embeddings API Documentation. https://platform.openai.com/docs/guides/embeddings

Oracle. (2026). Java SE 17 Documentation. https://docs.oracle.com/en/java/javase/17/

Spring. (2026). Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/docs/current/reference/html/

Spring. (2026). Spring Framework – JDBC Data Access. https://docs.spring.io/spring-framework/reference/data-access/jdbc.html

Thymeleaf. (2026). Thymeleaf Documentation. https://www.thymeleaf.org/documentation.html